

# Onboarding Problem

## 2D Heat Diffusion on a Uniform Grid

University of Waterloo High-Performance Computing Team

### Overview

---

You will implement two components of a 2D heat diffusion solver:

1. A **grid class** that stores the 2D temperature field in memory.
2. The **stencil kernel** that advances the simulation by one time step.

Everything else, i.e., the time integrator, boundary conditions, initial conditions, and I/O, is provided. You DO NOT need to know any thermal physics or theory.

### The Math

---

#### The Governing Heat Equation

Heat spreads across a 2D surface according to the following Partial Differential Equation:

$$\frac{\partial T}{\partial t} = \alpha \left( \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) \quad (1)$$

where  $T(x, y, t)$  is the temperature and  $\alpha$  is a constant. Again, you do not need to understand the physics, math, theory, or origin of the equation; this is merely for context.

#### Discretization onto a Grid

Replace the continuous derivatives with finite differences:

$$T_{t+1}(i, j) = T_t(i, j) + \Delta t \cdot \alpha \left( \frac{T_t(i+1, j) - 2T_t(i, j) + T_t(i-1, j)}{\Delta x^2} + \frac{T_t(i, j+1) - 2T_t(i, j) + T_t(i, j-1)}{\Delta y^2} \right) \quad (2)$$

#### Given Constants

With constants  $\alpha$ ,  $\Delta x$ ,  $\Delta y$ , and  $\Delta t$  chosen for stability, the equation simplifies to the following:

$$T_{t+1}(i, j) = 0.5 \cdot T_t(i, j) + 0.125 \cdot (T_t(i-1, j) + T_t(i+1, j) + T_t(i, j-1) + T_t(i, j+1)) \quad (3)$$

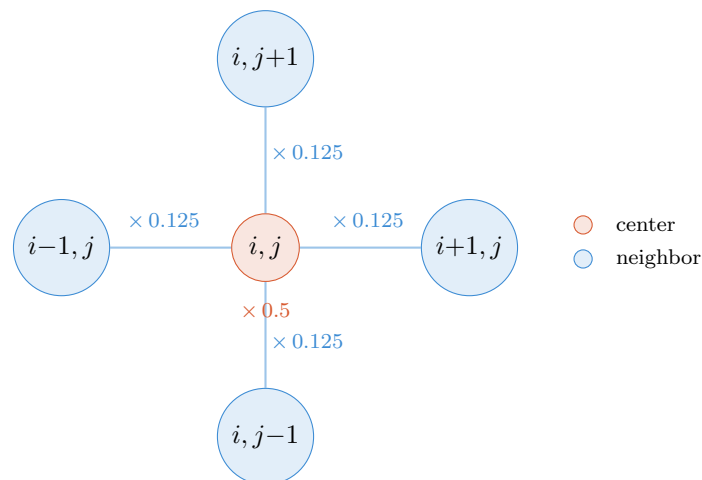
#### Converting Equations to Code

Notice how this is just a weighted average. In code, the above could look something like this:

```
1 new_arr[i][j] = 0.5 * old_arr[i][j] +
2               0.125 * (old_arr[i-1][j] +
3                       old_arr[i+1][j] +
4                       old_arr[i][j-1] +
5                       old_arr[i][j+1]
6                       );
```

## The Five-Point Stencil

To provide a more visual understanding and intuition of what is happening, consider the diagram below. You can picture the heat from each "bubble" (grid cell) leaking into its neighbors.



## Task 1: The Grid Class

Design and implement a `Grid` class that stores a 2D field of `double` values of size  $N_x \times N_y$ . While you may expand as much as you wish, your class must satisfy the following interface:

```
#include <cstddef>

class Grid {
private:
    std::size_t Nx_;
    std::size_t Ny_;

public:
    Grid(std::size_t nx, std::size_t ny);

    std::size_t Nx() const;
    std::size_t Ny() const;
};
```

The implementation of everything else is up to you. We want to see how you think and how you approach such problems.

### Requirements:

1. Default-initialized to zero.
2. You choose the memory layout. You will be asked to justify your choice.

## Task 2: The Stencil Kernel

Implement the function that applies the five-point stencil over all interior grid points ( $1 \leq i < N_x - 1$ ,  $1 \leq j < N_y - 1$ ).

Boundary values must remain unchanged. You are not required to implement any boundary conditions; simply copy the boundary values from the old grid to the new grid.

The pattern below is illustrative only; your Grid interface does not need to match it exactly.

```
void apply_stencil(const Grid& old_grid, Grid& new_grid){
    auto& new_cell{new_grid.cell()};
    const auto& old_cell{old_grid.cell()};

    // Copy boundaries
    for (std::size_t i{}; i < old_grid.Nx(); ++i) {
        new_cell[i][0] = old_cell[i][0];
        new_cell[i][old_grid.Ny() - 1] = old_cell[i][old_grid.Ny() - 1];
    }

    for (std::size_t j{}; j < old_grid.Ny(); ++j) {
        new_cell[0][j] = old_cell[0][j];
        new_cell[old_grid.Nx() - 1][j] = old_cell[old_grid.Nx() - 1][j];
    }

    // Your implementation...
}
```

## Deliverables

---

1. grid.hpp — the Grid class.
2. stencil.cpp — the apply\_stencil function.

## Evaluation

---

|                     |                                                            |
|---------------------|------------------------------------------------------------|
| <b>Correctness</b>  | Output matches reference within $10^{-10}$ per grid point. |
| <b>Performance</b>  | Wall-clock time on the team benchmark machine.             |
| <b>Code quality</b> | Clean C++17.                                               |

We will be selective in evaluating implementations. Note that correctness alone is not sufficient; we expect thoughtful consideration of performance, memory layout, and overall design.

## Submission

---

Prior to submission, we require a short write-up. This has no minimum or maximum length requirement; we just need you to justify your design choices. After your submission, we will invite you to a short virtual chat to discuss your design further.

## On Artificial Intelligence

---

We encourage using AI to learn and explore ideas. That said, your submission should be your own work; copy-pasting from AI will be obvious, especially during the chat.

Good luck.